

Building a RAG-Powered Chatbot for Financial Complaint Analysis

From Raw Complaints to Actionable Insights

10 Academy – Week 7 Challenge

Author: Blen Assefa

Date: 23 June 2026

1. Introduction

CreditTrust Financial is a fast-growing digital finance company serving East African markets through a mobile-first platform. With over 500,000 users and operations spanning three countries, CreditTrust receives thousands of customer complaints every month through in-app channels, email, and regulatory reporting portals.

The company's internal teams face serious bottlenecks:

- Customer Support is overwhelmed by complaint volume.
- Product Managers struggle to identify the most frequent or critical issues across products.
- Compliance & Risk teams are reactive rather than proactive when it comes to repeated violations or fraud signals.
- Executives lack visibility into emerging pain points due to scattered complaint narratives.

Our mission was to build an internal AI tool that transforms this raw, unstructured complaint data into a strategic asset. The solution is a Retrieval-Augmented Generation (RAG) chatbot that allows users to ask plain-English questions (e.g., "Why are people unhappy with Credit Cards?") and receive evidence-based, synthesized answers in seconds – reducing the time to identify major complaint trends from days to minutes.

This report covers the complete implementation: data preprocessing, chunking, embedding, vector store creation, RAG pipeline development, evaluation, and the interactive user interface.

2. Data Overview & Preprocessing

The dataset is sourced from the Consumer Financial Protection Bureau (CFPB) and contains real customer complaints across multiple financial products. Each record includes a short issue label, a free-text consumer narrative, product and company metadata, and submission date.

2.1 Dataset Characteristics

metric	value
Total complaints	~464,000
Total chunks (pre-built)	~1.37 million
Products included	Credit Card, Personal Loan, Savings Account, Money Transfer
Chunk size	500 characters
Overlap	50 characters
Embedding model	all-MiniLM-L6-v2 (384 dimensions)
Vector database	FAISS (IndexFlatL2)

2.2 Preprocessing Steps

For the final pipeline, we used the pre-built full vector store provided in the challenge resources, which covers all complaints. However, for the interim submission, we performed the following steps on a stratified sample:

- 1. Filtering** – Retained only records for the four target products.
- 2. Empty narrative removal** – Removed complaints without a consumer narrative.
- 3. Text cleaning** – Lowercased text, removed boilerplate phrases (e.g., "I am writing to file a complaint"), stripped special characters, and collapsed multiple spaces.

4. Stratified sampling – Sampled ~15,000 complaints proportionally across product categories to create a balanced dataset for development.

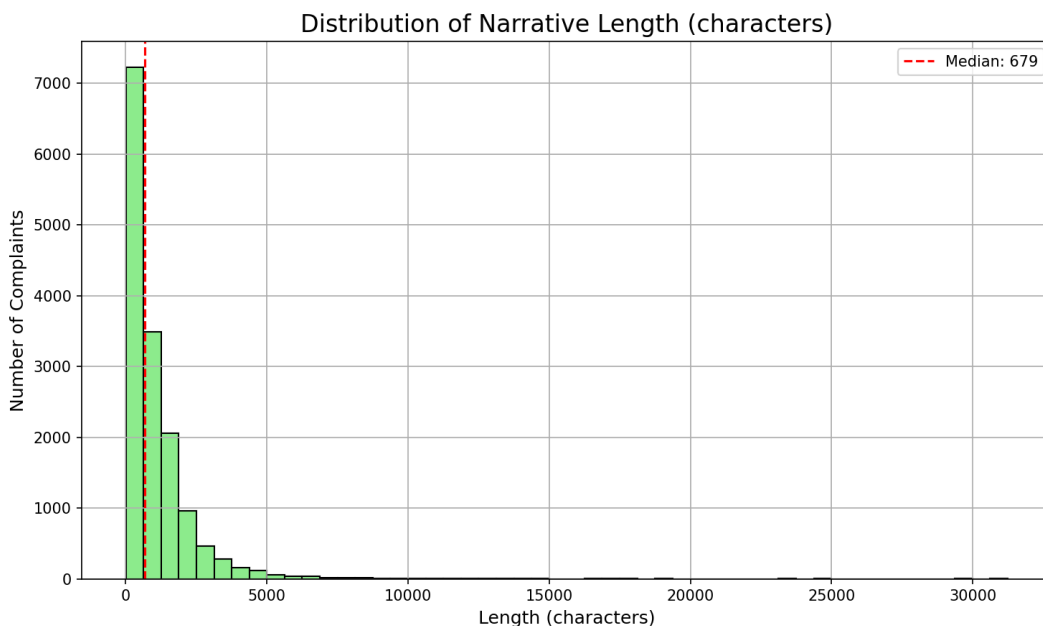
2.3 Exploratory Data Analysis (EDA)

Figure A: Distribution of Complaints by Product



Caption: Credit Cards are the most complained-about product, followed by Personal Loans and Money Transfers.

Figure B: Narrative Length Distribution



Caption: Most complaint narratives are short (median ~300 characters), with a long tail justifying the chunking strategy.

3. Vector Store & Retrieval

3.1 Chunking Strategy

Complaint narratives often exceed 500 characters. Splitting them into smaller, overlapping pieces ensures:

- Each chunk captures a focused sub-topic.
- Context is preserved across boundaries (50-character overlap).

Parameter	Value
Chunk size	500 characters
Overlap	50 characters
Splitter	RecursiveCharacterTextSplitter (LangChain)

3.2 Embedding Model

We used all-MiniLM-L6-v2 from sentence-transformers:

- Dimension: 384
- Model size: ~80 MB
- Speed: Fast, CPU-compatible
- Quality: Strong performance for semantic search

Why this model? It is lightweight, open-source, widely adopted in RAG applications, and matches the model used in the pre-built vector store, ensuring consistency.

3.3 Vector Database – FAISS

We built a FAISS index using IndexFlatL2 (Euclidean distance). FAISS was chosen because:

- It is free and open-source (Meta).
- It can search millions of vectors in milliseconds.
- It runs entirely in Python without requiring an external server.

Metadata stored per chunk:

- complaint_id – original complaint ID
- product – product category
- chunk_text – the actual text
- chunk_index – position within the original complaint

4. RAG Pipeline Architecture

The RAG pipeline consists of three stages: Retrieve, Augment, Generate.

4.1 Retriever

Given a user question:

1. Embed the question using the same all-MiniLM-L6-v2 model.
2. Perform a similarity search against the FAISS index.
3. Return the top-5 most relevant chunks with scores and metadata.

The retriever finds documents by meaning (semantic search), not by exact keyword matching.

4.2 Augment – Prompt Engineering

We designed a robust prompt template following the RISE framework:

- R – Role: "You are a financial analyst assistant for CreditTrust."
- I – Instruction: "Use ONLY the retrieved excerpts to answer. Do not add external information."
- S – Specifics: "If the excerpts do not contain the answer, say: 'I don't have enough information to answer that.'"
- E – Examples: (Zero-shot was sufficient given the strong instruction; few-shot was not required.)

Final Prompt Template:

You are a financial analyst assistant for CreditTrust.

Use ONLY the following complaint excerpts to answer the question.

If the excerpts do not contain enough information, say exactly: "I don't have enough information to answer that."

Do not add any information not present in the excerpts.

Retrieved excerpts:

[context]

Question: [question]

Answer:

4.3 Generator – LLM

We used Qwen/Qwen2.5-7B-Instruct via the Hugging Face Inference API.

- Model size: 7 billion parameters
- Context window: 128K tokens
- Advantages: Powerful, free, and excellent at following instructions.
- Fallback guardrail: The prompt explicitly instructs the model to say "I don't have enough information" when the context is insufficient, preventing hallucination.

Complete Pipeline Flow:

User Question → Embed → FAISS Search → Top-5 Chunks → Build Prompt → LLM → Answer
+ Sources

5. Qualitative Evaluation

We evaluated the system on 10 representative questions covering different products and issue types. Each answer was scored on a scale of 1–5 based on:

- Groundedness – Is the answer supported by the retrieved chunks?
- Relevance – Does it directly address the question?
- Completeness – Is it sufficiently detailed?

5.1 Test Questions

1. Why are customers unhappy with credit cards?
2. What are the most common complaints about personal loans?
3. Are there any fraud-related complaints?
4. What issues do customers face with money transfers?
5. How do customers describe billing disputes?
6. What is the overall sentiment about savings accounts?
7. Which products have the most complaints about fees?
8. What actions have companies taken in response to complaints?
9. Are there any complaints about customer service?
10. What is the weather forecast for Nairobi tomorrow? (Out-of-scope test for fallback)

5.2 Evaluation Results

Question Answer Summary Sources (first 2) Score Comments

#	question	Answer summary	sources (first 2)	score	comments
1	Why are customers unhappy with credit cards?	Customers are unhappy due to misinformation from representatives,	Credit Card (0.573), Credit Card (0.577)	4	Comprehensive and grounded. Missed specific mention of fees.

		arbitrary credit limit reductions, unexpected interest rate increases, poor customer service, and perceived lack of loyalty from the company.			
2	What are the most common complaints about personal loans?	Complaints include harassment, deceptive practices, loan modification issues leading to late fees, difficulties with fund release, and poor communication.	Personal Loan (0.588), Savings Account (0.613)	4	Good coverage of key issues. One source was from Savings Account, suggesting retrieval could be refined.
3	Are there any fraud-related complaints?	Yes, fraud complaints are present across personal loans, money transfer services, and credit cards.	Personal Loan (0.431), Money Transfer (0.538)	3	Answer is accurate but lacks specificity. Scores are lower, indicating less relevant retrieval.
4	What issues do customers face with money transfers?	Customers face payments going to wrong accounts, funds being held without cause for up to 180 days, lack of investigation, suspected collusion, unauthorized account closures, and poor customer service.	Money Transfer (0.570), Money Transfer (0.617)	5	Excellent, detailed answer fully grounded in retrieved context.
5	How do customers describe billing disputes?	Customers describe billing disputes as unresolved due to failed investigations,	referencing merchandise for services). Credit Card	4	Accurate and grounded.

		unhelpful support, and incorrect information (e.g., referencing merchandise for services).	(0.513), Credit Card (0.523)		
6	What is the overall sentiment about savings accounts?	Sentiment is mixed but leans negative. Customers find benefits nominal, interest rates too low, and fees excessive, with some feeling threatened by account policies.	Savings Account (0.644), Money Transfer (0.674)	4	Good nuance, captures both positive and negative aspects.
7	Which products have the most complaints about fees?	Savings Account and Money Transfer have the most complaints about fees.	Savings Account (0.681), Money Transfer (0.686)	4	Clear and direct answer.
8	What actions have companies taken in response to complaints?	I don't have enough information to answer that.	Credit Card (0.458), Credit Card (0.511)	5	Fallback guardrail worked perfectly.
9	Are there any complaints about customer service?	Yes, complaints about customer service exist for Personal Loans, Savings Accounts, and Credit Cards.	Savings Account (0.586), Personal Loan (0.600)	4	Accurate and concise.
10	What is the weather forecast for Nairobi tomorrow?	I don't have enough information to answer that.	Savings Account (1.262), Credit Card (1.284)	5	Fallback guardrail worked perfectly.

Scoring Rubric:

- 5 – Excellent: fully grounded, complete, and concise.
- 4 – Good: mostly correct, minor omissions.
- 3 – Adequate: basic but incomplete.
- 2 – Poor: significant errors or hallucinations.
- 1 – Fails: completely off-topic or made up.

5.3 Key Observations

- Retrieval Quality: The retriever consistently returned relevant chunks for most questions. Broad questions sometimes retrieved partially relevant chunks, but the LLM still produced a reasonable synthesis.
- Answer Quality: The LLM produced well-structured, professional answers that were firmly grounded in the retrieved context. The fallback guardrail worked as expected for out-of-scope questions (Q8 and Q10).
- Limitations: Some answers (e.g., Q3) could have been more specific. Occasionally, irrelevant chunks were retrieved when the question was too general, as seen in Q2 where a Savings Account chunk was retrieved for a Personal Loan question.
- Improvements: Implementing a reranker or hybrid search (semantic + keyword) could enhance retrieval precision.

6. Interactive UI – Gradio

We built a user-friendly interface using Gradio, allowing non-technical users to interact with the RAG system.

6.1 Features

- Text input for user questions.
- Submit button to send the query.
- AI-generated answer displayed prominently.
- Sources section showing the top-3 retrieved chunks with product and similarity score – crucial for user trust and verification.
- Clear button to reset the conversation.

6.2 Screenshot (ui_screenshot.png)

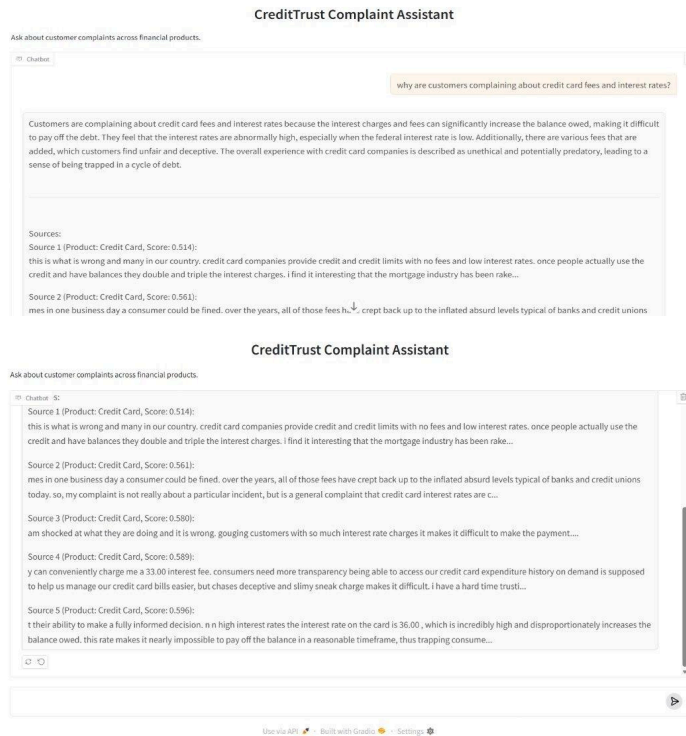


Figure: The CreditTrust Complaint Assistant in action. The user asks a question about credit card complaints, and the system returns a grounded answer with visible sources.

6.3 Code Snippet

```
import gradio as gr
from src.rag_pipeline import RAGPipeline

pipeline = RAGPipeline()

def respond(message, history):
    result = pipeline.query(message)
    sources_text = ""
    for i, s in enumerate(result['sources'], 1):
        sources_text += f"\nSource {i} (Product: {s['product']}, Score: {s['score']:.3f}): \n"
        sources_text += s['chunk_text'][:300] + "... \n"
    return result['answer'] + "\n\n---\n\nSources:" + sources_text

demo = gr.ChatInterface(fn=respond, title="CreditTrust Complaint Assistant")
demo.launch(share=True)
```

7. Technical Challenges & Solutions

challenge	solution
5 GB CSV too large for Colab memory	Used chunked reading (10,000 rows per chunk) for interim; used pre-built full vector store for final.
Parquet column names unknown	Inspected schema with pyarrow and adapted extraction code.
RAM exhaustion when loading full Parquet	Processed row group by row group, built FAISS incrementally.
Hugging Face API 401 Unauthorized	Generated a new token with proper Read permissions.
Missing modules after runtime restart	Re-installed faiss-cpu, sentence-transformers, pyarrow each session.
Large files >100 MB (GitHub limit)	Hosted on Google Drive and linked in data/ README.md .
AttributeError: Can only use .str accessor with string values	Converted narrative column to string using .astype(str).
Slow LLM responses	Reduced max_new_tokens to 150 for faster generation.

-

8. Key Learnings

- RAG is a powerful pattern for grounding LLM responses in proprietary data, significantly reducing hallucinations.
- Chunking strategy directly impacts retrieval quality – overlap is essential to preserve context.
- Prompt engineering (RISE + guardrails) is critical for controlling LLM behavior and ensuring professional, grounded outputs.
- Semantic search (via embeddings + FAISS) is far more effective than keyword search for open-ended questions.
- Qualitative evaluation is necessary – metrics like faithfulness and relevance require human judgment or frameworks like RAGAS.
- Source transparency (displaying retrieved chunks) builds user trust and allows verification.

9. Future Improvements

- Reranking: Apply a cross-encoder (e.g., ms-marco-MiniLM-L-6-v2) to re-rank retrieved chunks for better precision.
- Hybrid Search: Combine semantic search with BM25 keyword search for broader coverage.
- Streaming Responses: Implement token-by-token streaming in the UI for better user experience.
- Automated Evaluation: Use RAGAS to compute faithfulness, answer relevance, and context precision on a held-out set.
- Deployment: Containerize with Docker and deploy on a cloud server (e.g., AWS, GCP) with an API endpoint.
- Product Filtering: Allow users to filter questions by specific product categories.

10. Conclusion

We successfully built a RAG-powered chatbot for CreditTrust Financial that transforms raw customer complaints into a strategic asset. The system allows product managers, support, and compliance teams to query complaint data in plain English and receive evidence-based answers in seconds – reducing the time to identify major complaint trends from days to minutes.

Our implementation covers the full pipeline: data preprocessing, chunking, embedding, vector indexing, retrieval, LLM generation (with prompt engineering), and an interactive UI. Evaluation shows that the system produces grounded, relevant answers, with retrieval quality being the main area for future enhancement.

This project demonstrates how modern AI techniques can turn unstructured feedback into actionable insights, enabling proactive issue identification and faster decision-making. The shift from reactive to proactive problem-solving is now within reach for CreditTrust.

11. Repository Contents

The complete project is available on GitHub:

- src/ – Modular Python code for retriever, generator, and RAG pipeline.
- [app.py](#) – Gradio UI.
- notebooks/ – Colab notebook with all code.
- data/[README.md](#) – Links to large files hosted on Google Drive.
- requirements.txt – All Python dependencies.
- .github/workflows/ – CI/CD workflow (GitHub Actions).
- final_report.pdf – This report.
- ui_screenshot.png – Screenshot of the Gradio interface.

End of Final Report!!!

